

CODED - Cloud-Optimized Distributed Earth Data

Can the decentralized peer-to-peer **InterPlanetary File System (IPFS)** make important environmental datasets resilient against institutional takedowns, while remaining useful for cloud-native geospatial workflows? (Zarr · Icechunk · xarray)

Cory Levinson · Rich Signell · Brian Terry

★ An ESIP Funding Friday Project

Objective

The geoscience community has converged on **cloud-optimized formats** - Zarr, and now **Icechunk** - that stream huge datasets via chunk-level range requests. **IPFS** offers content-addressed, location-independent, takedown-resistant storage. Can the two be married?

This round we go past "does it work" to "where does the time go" - separating discovery, transfer, and caching costs.

Preserve *your* dataset - CID on Filebase

Have an existing **Zarr** or **Icechunk** store you can't afford to lose? Add it to IPFS for a content ID (CID), then pin that CID on **Filebase** (IPFS + Filecoin, free to 5 GB / 500 pins) - or **any public IPFS node** - so it stays available, verifiable, and independent of any single institution.

```
# 1. add your existing store to IPFS → root CID
cid=$(ipfs add -rQ --cid-version=1 my.icechunk/)
# 2. export the whole DAG (all chunks + refs) to a CAR file
ipfs dag export $cid > my.car
# 3. pin that CAR on Filebase — import=car preserves the CID
aws s3 cp my.car s3://bucket/my.car \
  --endpoint-url https://s3.filebase.com \
  --metadata import=car # same CID, now persistent
```

The **CID is a hash of the data** — tamper-evident and location-independent. Our live ERA5 store:

bafybeiecltl3mtags2i3jeumxe5c7zuvj2r76zxwsg6tfljiouvvywqzbaq

One CID = the whole queryable store. Re-chunked to 100 blocks (5 timesteps each) to fit the free-tier pin budget - **data byte-identical** (maxdiff 0.0).

Icechunk reads IPFS - no adapter

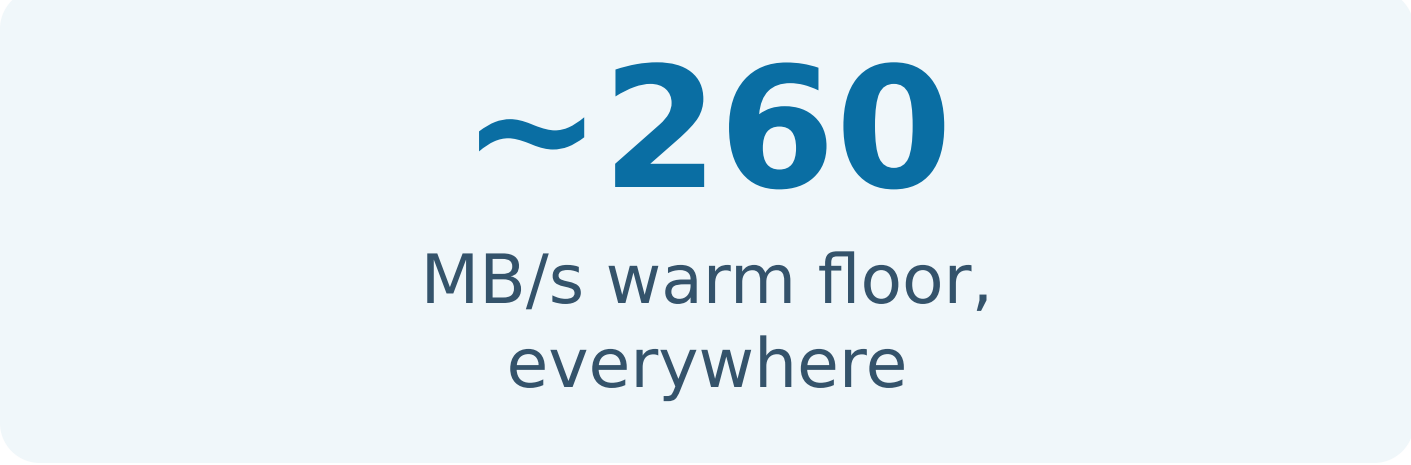
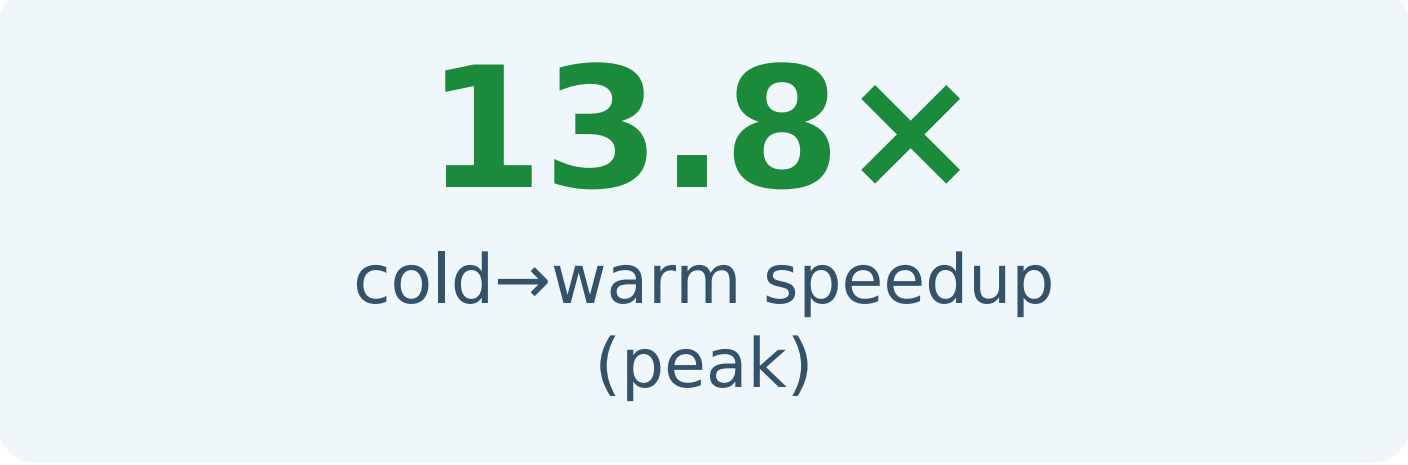
Icechunk 2.0.5 ships `http_storage(base_url=...)`, a read-only store over HTTP. Point it at an IPFS gateway path and the entire repo opens in xarray - the on-disk layout survives `ipfs add -r` untouched.

```
# whole Icechunk repo, straight from a CID
st = icechunk.http_storage("https://gw/ipfs/<CID>")
repo = icechunk.Repository.open(st)
sess = repo.readonly_session("main")
ds = xr.open_zarr(sess.store)
```

Old CID → old snapshot (reproducibility). Edit one of four chunks → **6 shared blocks reused** — cross-version dedup for free.

Caching is the real win

Once a block has been fetched, the second read serves from the local blockstore - no network round-trip. With **100 fat chunks (~21 MB)** the cold→warm gains are even sharper than the old 500-chunk layout.



Warm reads converge to **~8 s / ~260 MB/s** regardless of topology. A local pin **is** the warm floor - and here it even **beats native Icechunk-on-S3**.

Where does the time go?

We read the **same 2.08 GB ERA5** 2-m temperature slice - now **re-chunked to 100 blocks** (5 timesteps each) - via **Icechunk over IPFS** (`http_storage`) three ways, in two topologies, against a **native Icechunk-on-S3** control. Every path returns a bit-identical mean = **286.5062 K**.

Cold read path	Same-region	Cross-region
Local pin (warm floor)	~7.6 s	~7.9 s
Direct peer (Bitswap)	+103 s	+82 s
DHT discovery vs direct	~noise	~noise
Icechunk-on-S3 (control)	~14 s	~14 s

Transfer overhead = direct - local (a full 2 GB bitswap re-fetch per cold cell). Discovery (DHT vs direct) is within noise in a 2-node swarm - a lower-bound caveat, not a general claim. S3-IC = native Icechunk-on-S3 of the identical rechunked bytes.

What This Means

- **Blame the distance, not the DHT.** Discovery is cheap; a full 2 GB single-peer bitswap re-fetch is the cost. More peers & regional gateways close the gap.
- **Warm ≈ free.** Caching / pinning gives repeat readers a **~8 s / ~260 MB/s** local floor - and a local pin now **beats native Icechunk-on-S3** (~14 s cold).
- **Icechunk-on-IPFS is real today** via `http_storage` - open the CID straight into xarray, reproducible & dedup-friendly, zero adapter code.

Bottom Line
Distance dominates cold reads - not discovery. Finding content on IPFS is cheap; the first cross-continent 2 GB pull is the cost. But **pin locally and IPFS-Icechunk is the fastest path here** - ~8 s warm, beating native Icechunk-on-S3 - with verifiable, takedown-resistant, content-addressed data.

